Deep Learning - COSC2779/2972

Transformers & Self-Attention

A/Prof. Ruwan Tennakoon

RMIT UNIVERSITY

Reference: *Vaswani et al., "Attention Is All You Need", NeurIPS 2017.*

Outline

1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

Last weeks – Recurrent models:

- Process a sequence one token at a time, carrying a hidden state $h^{\langle t \rangle}$.
- GRU / LSTM: gating to capture long-range dependencies.
- Encoder–decoder + **attention** for sequence-to-sequence tasks.

Last weeks – Recurrent models:

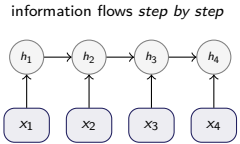
- Process a sequence one token at a time, carrying a hidden state $h^{\langle t \rangle}$.
- GRU / LSTM: gating to capture long-range dependencies.
- Encoder–decoder + **attention** for sequence-to-sequence tasks.

The key idea we keep: attention lets the decoder look back at *all* encoder states, not just the last one.

Last weeks – Recurrent models:

- Process a sequence one token at a time, carrying a hidden state $h^{\langle t \rangle}$.
- GRU / LSTM: gating to capture long-range dependencies.
- Encoder–decoder + **attention** for sequence-to-sequence tasks.

The key idea we keep: attention lets the decoder look back at *all* encoder states, not just the last one.



Objectives of this lecture

- Understand **why** we move beyond recurrence: the limitations of RNNs at scale.
- Build up the **self-attention** mechanism from first principles:
  - Queries, Keys, Values and scaled dot-product attention.
  - Multi-head attention.
- Understand how **positional encoding** restores order information.
- Assemble the pieces into a full **Transformer encoder block**, and see how the family (BERT, GPT, ViT) is built from it.

Outline

1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

Recap: the recurrent encoder

A recurrent encoder reads the sequence and compresses it into hidden states:

$$h^{\langle t \rangle} = g\left(W_{hh}\, h^{\langle t-1 \rangle} + W_{hx}\, x^{\langle t \rangle}\right).$$

The good:

- Order is built in "for free" through recurrence.
- Cost grows *linearly* with sequence length.
- Works online – no need to see the future.

The bad:

- Must process tokens **one at a time** – cannot parallelise over time.
- Long-range signal must survive many steps – it tends to **wash out**.
- The whole sequence is squeezed through a single hidden-state "bottleneck".

"What if a token could look directly at every other token?"

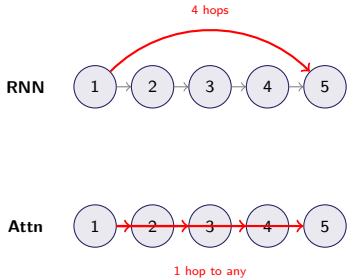
Path length: why long-range is hard

In an RNN, for token 1 to influence token n, the signal travels through $n-1$ intermediate states.

Maximum path length between two positions:

- RNN: $\mathcal{O}(n)$ – grows with the sequence.
- Self-attention: $\mathcal{O}(1)$ – every token connects to every other **directly**.

Shorter paths $\Rightarrow$ gradients survive $\Rightarrow$ long-range dependencies are learnable.



Per layer, comparing a sequence of length n with representation size d:

Layer type	Complexity / layer	Sequential ops	Max path length
Recurrent	$\mathcal{O}(n\,d^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Self-attention	$\mathcal{O}(n^2\,d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

- Self-attention trades a **quadratic** dependence on length (n^2) for **constant** sequential depth.
- On modern hardware, "all tokens at once" maps beautifully onto GPUs.
- The catch: n^2 memory/compute – the reason very long sequences are still hard (and an active research area).

Outline

1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

The intuition

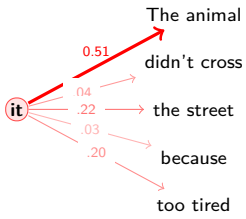
Read: *"The animal didn't cross the street because **it** was too tired."*

What does **"it"** refer to? To resolve it, the word must *look at* the rest of the sentence and decide which words matter.

Self-attention lets every token:

1. look at all other tokens,
2. score how relevant each one is,
3. pull in a weighted blend of their information.

The output for each token is a **context-aware** version of its embedding.



attention weights for query "it"

Start simple. Represent each token by its embedding $x^{\langle i \rangle} \in \mathbb{R}^d$.

A natural measure of "how relevant is token j to token i" is a **similarity** between their embeddings. The **dot product** is the workhorse choice:

$$e_{ij} = x^{\langle i \rangle \top} x^{\langle j \rangle}.$$

Large when the vectors point the same way; near zero when orthogonal.

Turn the scores for query i into weights that sum to one with a **softmax**:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}.$$

The new representation is a weighted average of the others:

$$z^{\langle i \rangle} = \sum_j \alpha_{ij} \, x^{\langle j \rangle}.$$

Problem

Using the raw embedding for all three roles is too rigid – a token plays the same part whether it is *asking*, *being matched*, or *contributing content*.

Fix: give each role its own learned projection.

From each embedding $x^{\langle i \rangle}$ we compute three vectors via learned matrices:

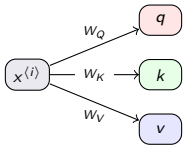
$$\underbrace{q^{\langle i \rangle} = W_Q\, x^{\langle i \rangle}}_{\text{Query}}, \quad \underbrace{k^{\langle i \rangle} = W_K\, x^{\langle i \rangle}}_{\text{Key}}, \quad \underbrace{v^{\langle i \rangle} = W_V\, x^{\langle i \rangle}}_{\text{Value}}.$$

A useful analogy – a **dictionary lookup**:

- **Query**: what I am looking for.
- **Key**: what each token advertises.
- **Value**: the content each token returns *if* it is attended to.

Score = query against every key; weights = softmax; output = weighted sum of values:

$$\begin{aligned} e_{ij} &= q^{\langle i \rangle \top} k^{\langle j \rangle}, \\ \alpha_{ij} &= \mathrm{softmax}_j(e_{ij}), \\ z^{\langle i \rangle} &= \sum_j \alpha_{ij}\, v^{\langle j \rangle}. \end{aligned}$$



Stack the vectors as rows of matrices Q, K, V (one row per token). The whole operation is a few matrix products:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- $QK^\top$: all query–key scores at once, an $n \times n$ matrix.
- softmax is applied **row-wise** (each query normalises over keys).
- multiply by V: each output row is a blend of value rows.

Why divide by $\sqrt{d_k}$?

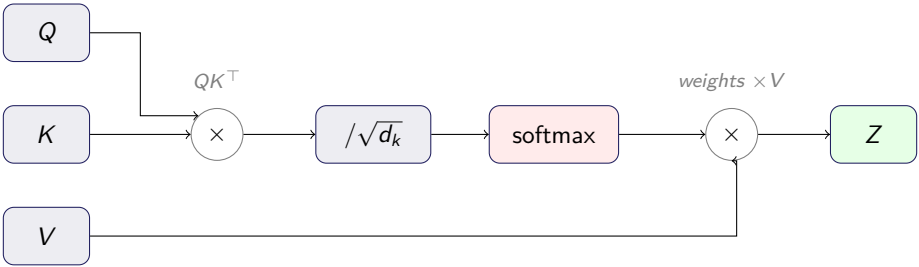
If q, k have components with unit variance, then

$$q^\top k = \sum_{m=1}^{d_k} q_m k_m$$

has variance $\approx d_k$. For large d_k the scores get large, softmax **saturates**, and gradients vanish. Dividing by $\sqrt{d_k}$ keeps the variance ≈ 1.

A small fix, but it matters a lot for stable training.

The computation, end to end



- Every arrow is a plain tensor op – no loops over time, so the whole sequence is processed **in parallel**.
- The only learned parameters so far are W_Q, W_K, W_V (inside Q, K, V). The attention step itself has *no* weights.

Check your understanding

1. In $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$, what are the dimensions of each matrix for n tokens, key size d_k and value size d_v?
2. If we removed the softmax and just used the raw scores, what would break?
3. Self-attention is *permutation-equivariant*: shuffle the input tokens and the outputs shuffle the same way. Why is that a **problem** for language, and what will we add to fix it?

Outline

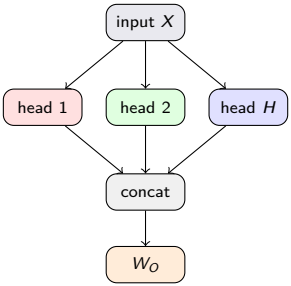
1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

One attention is not enough

A single attention layer produces **one** weighting per query. But a word can relate to others in *several* ways at once:

- syntactic role (subject $\leftrightarrow$ verb),
- coreference ("it" $\leftrightarrow$ "animal"),
- local vs. long-range context.

Idea: run several attention operations in parallel – "heads" – each with its own W_Q, W_K, W_V, so each can specialise. Then combine them.



Multi-head attention: the math

For H heads, each head h gets its own projections:

$$\text{head}_h = \text{Attention}\Big(XW_Q^h,\, XW_K^h,\, XW_V^h\Big).$$

Concatenate the heads and mix with a final projection W_O:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, W_O.$$

Typically each head is made **narrow**: $d_k = d_v = d/H$. So H heads cost about the same as one full-width attention.

Dimensions (worked)

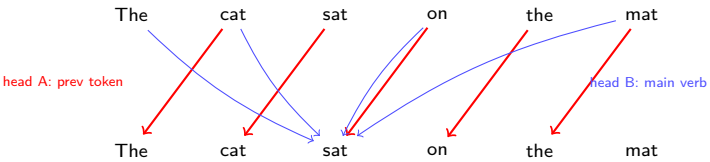
Model width $d = 512$, $H = 8$ heads
$\Rightarrow d_k = d_v = 64$.

- $W_Q^h, W_K^h, W_V^h : 512 \times 64$
- each $\text{head}_h : n \times 64$
- concat: $n \times 512$
- $W_O : 512 \times 512$, output $n \times 512$

Output shape matches the input – so we can **stack** these layers.

What do heads learn?

Different heads tend to specialise – when we visualise their attention weights, some track the previous token, some attend to the verb, some resolve references.



Important: this specialisation *emerges* from training – we never tell a head what to attend to.

Outline

1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

The order problem

Self-attention is a weighted sum over a **set**: it has no notion of *where* a token sits. So

$$\text{“dog bites man”} \equiv \text{“man bites dog”}$$

would produce the same representations. For language (and time series) that is clearly wrong.

Naive fixes that fail:

- Append the raw index t: magnitude grows without bound.
- Append t/n (normalised): the same position means different things for different sequence lengths.

The fix

Add a **positional encoding** vector $p^{\langle t \rangle} \in \mathbb{R}^d$ to each token embedding:

$$\tilde{x}^{\langle t \rangle} = x^{\langle t \rangle} + p^{\langle t \rangle}.$$

Same dimension as the embedding, so it flows through unchanged.

We want $p^{\langle t \rangle}$ to be bounded, unique per position, and to make *relative* offsets easy to represent.

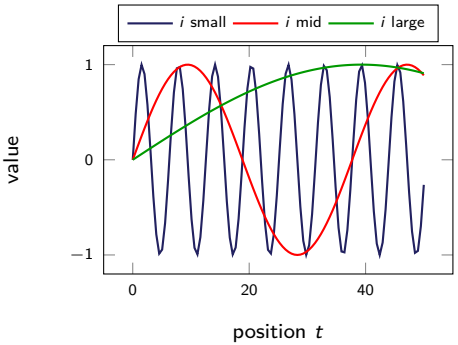
Sinusoidal positional encoding

The original Transformer uses fixed sinusoids of geometrically increasing wavelength. For position t and dimension index i:

$$p_{2i}^{\langle t \rangle} = \sin\left(\frac{t}{10000^{2i/d}}\right),$$

$$p_{2i+1}^{\langle t \rangle} = \cos\left(\frac{t}{10000^{2i/d}}\right).$$

- Low dimensions oscillate **fast**, high dimensions oscillate **slowly** – like the bits of a binary counter.
- Bounded in $[-1, 1]$, and defined for any length.
- $p^{\langle t+\Delta \rangle}$ is a **linear** function of $p^{\langle t \rangle}$ – so relative shifts are easy to learn.



Different dimensions = different frequencies.

- **Absolute** (what we just built): each position gets a fixed vector, added at the input. Simple; can be *fixed* (sinusoidal) or *learned*.
- **Relative**: encode the *distance* $t_i - t_j$ between a query and key directly inside the attention score. What matters for language is usually "how far apart", not "absolute index".

Why relative encodings are attractive

They depend only on pairwise distance, so a model trained on short sentences can **generalise** to longer sequences it never saw at training time. Variants (RoPE, ALiBi, T5 biases) power most modern large language models.

Outline

1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

Assembling a block

Attention mixes information *across* tokens. We wrap it with two more ingredients that act *per token*:

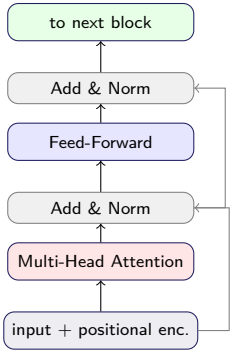
- **Feed-forward network** (FFN): a small MLP applied to each position independently –

$$\mathrm{FFN}(z) = \max(0,\, zW_1 + b_1)\, W_2 + b_2.$$

  This is where most of the per-token "thinking" (and most parameters) live.

- **Residual + LayerNorm** around each sub-layer:

$$\mathrm{out} = \mathrm{LayerNorm}(z + \mathrm{Sublayer}(z)).$$



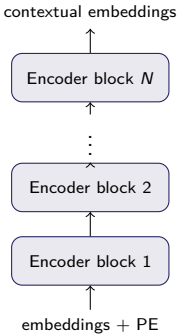
Stack them – "Transformers"

One block outputs a sequence of the **same shape** as its input. So we stack N of them (the paper uses $N = 6$; large models use dozens).

Each block *transforms* every token's embedding using the whole sequence as context – hence the name.

A few consequences:

- No recurrence and no convolution – attention is the only mixing operation.
- Bi-directionality is automatic: every token sees left *and* right context, so no separate "Bi-" variant is needed.
- A special **[CLS] / class token** can aggregate a global summary of the sequence – handy for classification (akin to the final RNN state).



Self-attention in a few lines of PyTorch

```python
import torch, torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q,K,V: (batch, heads, seq, d_k)
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1)    # (.., seq, seq)
    scores = scores / d_k ** 0.5        # scale
    if mask is not None:                # optional causal mask
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return weights @ V                  # (.., seq, d_k)
```

- The entire mechanism is two matrix multiplies around a softmax.
- `mask` is how a **decoder** forbids attending to future tokens.
- Multi-head = reshape the last dim into `(heads, d_k)`, run this, then merge back.

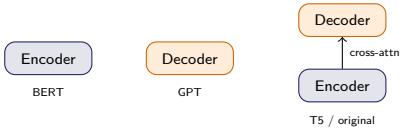
Outline

1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

Three ways to use the block

The same encoder block gives **three** model families, differing only in *masking* and *which halves are present*:

- **Encoder-only** (BERT): bi-directional – every token sees the whole sequence. Best for *understanding*.
- **Decoder-only** (GPT): masked – token t sees only $\leq t$. Best for *generation*.
- **Encoder–decoder** (T5, the original): encoder reads the input, decoder generates while attending back to it. Best for *seq-to-seq*.



The building block never changes – only how we wire and mask it.

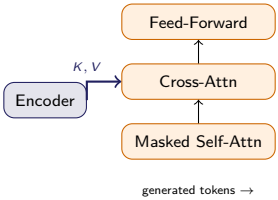
The decoder: masking + cross-attention

A decoder block adds two things to the encoder block:

- **Masked self-attention.** During generation, token t must not peek at future tokens. We add a mask that sets scores for $j > t$ to $-\infty$ *before* the softmax:

$$\tilde{e}_{ij} = \begin{cases} e_{ij} & j \le i \\ -\infty & j > i \end{cases}$$

- **Cross-attention** (encoder–decoder only). Queries come from the decoder; **keys and values come from the encoder output** – this is how the decoder "reads" the source sentence.



Generation is **autoregressive**: predict one token, append it, repeat.

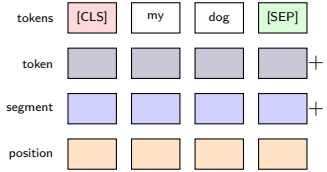
BERT: input representation

BERT is an encoder-only model. Its input is built from **three** embeddings, *summed* per position:

$$\text{input}_t = \underbrace{E_t^{\text{tok}}}_{\text{which word}} + \underbrace{E_t^{\text{seg}}}_{\text{sentence A/B}} + \underbrace{E_t^{\text{pos}}}_{\text{position}}$$

Two special tokens are added:

- **[CLS]** at the front – a slot whose final embedding summarises the *whole* sequence.
- **[SEP]** to separate / terminate sentences.



Each column is one position; the three embeddings add to a single input vector.

BERT: what comes out, and how it is trained

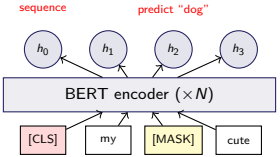
Run the sequence through the encoder stack. For every input position we get a **contextual embedding**:

- the **[CLS]** output → a single vector representing the whole sequence,
- each **token** output → a context-aware embedding of that word.

Pre-training (self-supervised, no labels):

- **Masked LM**: hide ∼15% of tokens, predict them from both-side context.
- **Next-Sentence Prediction**: does sentence B follow A? (uses the [CLS] vector).

Then **fine-tune** on a small labelled dataset for the actual task.



[CLS] → whole-sequence vector; the masked position is trained to reconstruct the hidden word.

Outline

1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

Sequence classification – use the [CLS] token

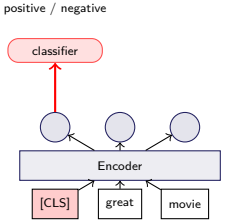
Tasks with **one label per sequence**: sentiment, spam, topic, natural-language inference.

Recipe:

1. Encode the sequence.
2. Take the **[CLS]** output vector $h_{\texttt{[CLS]}}$ (the global summary).
3. Feed it to a small classifier head:

$$\hat{y} = \mathrm{softmax}(W\, h_{\texttt{[CLS]}} + b).$$

Only the head (and, when fine-tuning, the encoder) is trained on labels.



Only the [CLS] output feeds the head.

Token classification – use every token embedding

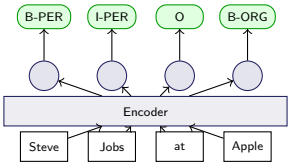
Tasks with **one label per token**: named-entity recognition (NER), part-of-speech tagging, span extraction.

Recipe:

1. Encode the sequence.
2. Take **every** token's output embedding h_t.
3. Apply the *same* head to each position:

$$\hat{y}_t = \mathrm{softmax}(W\, h_t + b).$$

The embeddings are contextual, so "Washington" can be tagged PERSON or LOCATION depending on its neighbours.



One shared head, applied at every position.

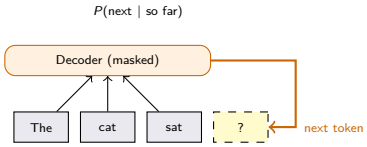
Generation – use a decoder

Tasks that **produce a sequence**: language modelling, translation, summarisation, chat.

Recipe (autoregressive):

1. Decoder with **masked** self-attention predicts a distribution over the next token.
2. Pick one (greedy / sampling / **beam search** – recall Week 7).
3. Append it to the input and repeat until `[EOS]`.

Encoder–decoder models add cross-attention so generation is *conditioned* on a source (translation); decoder-only models (GPT) just continue the prompt.



Each step attends only to tokens already produced.

Outline

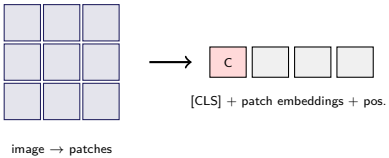
1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

An image as a sequence of patches

Self-attention needs a *sequence*. An image is a grid of pixels – too many to treat each pixel as a token (n^2 attention would explode). The **Vision Transformer (ViT)** idea: cut the image into patches and treat each patch as a "word".

Turning an image into tokens:

1. Split into fixed 16×16 patches.
2. Flatten each patch ($16 \times 16 \times 3 = 768$ values) and **linearly project** it to the model width – these are the patch embeddings.
3. Prepend a learnable **[CLS]** token.
4. Add **positional encodings** (patches have a 2-D location).

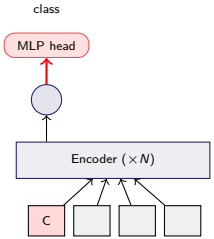


"An image is worth 16×16 words" – Dosovitskiy et al., ICLR 2021.

How ViT works (and its catch)

Once the image is a sequence, it is just a **standard Transformer encoder**:

1. Feed the patch sequence through N encoder blocks.
2. Read the final **[CLS]** embedding.
3. An MLP head maps it to class scores – exactly the sequence-classification recipe from earlier.



The catch: weak inductive bias

A CNN bakes in *locality* and *translation equivariance*. ViT assumes almost nothing – it must *learn* spatial structure from data. So it is **data-hungry**: it only beats CNNs when pre-trained on very large datasets. **DeiT** narrows this gap with strong augmentation and *distillation* from a CNN teacher.

Outline

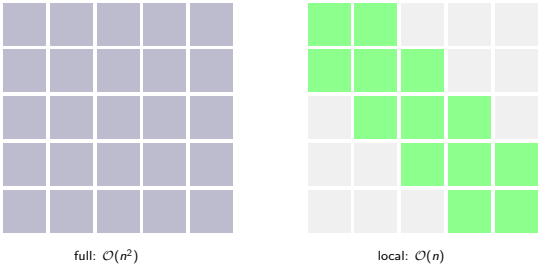
1. Motivation: Beyond Recurrence
2. Self-Attention
3. Multi-Head Attention
4. Positional Encoding
5. The Transformer Encoder Block
6. The Transformer Family
7. Using Transformers for Downstream Tasks
8. Vision Transformers
9. Beyond Vanilla Self-Attention

The efficiency problem

Vanilla self-attention builds the full $n \times n$ score matrix, so compute and memory grow as $\mathcal{O}(n^2)$. For long documents, high-resolution images or genomes, that is the binding constraint.

Directions:

- **Sparse / local** attention – each token attends to a window or a few global tokens (Longformer, BigBird): $\mathcal{O}(n)$.
- **Linear / kernel** attention – approximate the softmax to avoid the $n \times n$ matrix (Performer, Linformer).
- **IO-aware exact** attention – **FlashAttention**: same maths, far less memory traffic; now standard.



Restricting *which* pairs interact is the main lever.

Architectural refinements

The block introduced by Vaswani et al. has been steadily refined – most modern LLMs share these changes:

Component	Vanilla (2017)	Modern practice
Positional info	absolute sinusoids, added at input	**relative**: RoPE, ALiBi – extrapolate to longer contexts
Normalisation	LayerNorm *after* sub-layer (post-LN)	**pre-LN** + RMSNorm – more stable at depth
Feed-forward	ReLU MLP	**gated** activations (GLU / SwiGLU)
Attention heads	full multi-head	**MQA / GQA** – share keys/values for fast inference
Capacity	one dense FFN	**Mixture-of-Experts** – route each token to a few experts

- The theme: same self-attention core, but cheaper, more stable, and easier to scale.
- Scaling this recipe – more data, parameters and compute – is what produced today's **foundation models** (GPT, LLaMA, ViT-based vision models).

Summary

- Self-attention – $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$ with multi-head projections – replaces recurrence: $\mathcal{O}(1)$ path length, full parallelism.
- **Positional encoding** restores order; a **block** = attention + FFN wrapped in residual + LayerNorm, stacked N times.
- One block, three families: **encoder-only** (BERT, bi-directional), **decoder-only** (GPT, masked), **encoder–decoder** (cross-attention).
- **BERT** input = token + segment + position embeddings, with a **[CLS]** summary token.
- Downstream: **[CLS]** $\rightarrow$ sequence classification; **per-token** embeddings $\rightarrow$ NER / tagging; **decoder** $\rightarrow$ generation.
- **ViT** turns an image into a sequence of patch embeddings and runs the same encoder – powerful but data-hungry.
- Beyond vanilla: efficient attention ($\mathcal{O}(n^2) \rightarrow$ sparse/linear/FlashAttention), relative positions, pre-LN/RMSNorm, GQA, MoE.

Next week: Self-supervised learning & foundation models.